AI ASSISTED CODING

ASSIGNMENT-9.5

NAME: Geethanjali

HTNO: 2303A51005

Problem 1: String Utilities Function

Consider the following Python function: def

reverse_string(text):

return text[::-1]

Task:

1. Write documentation in:

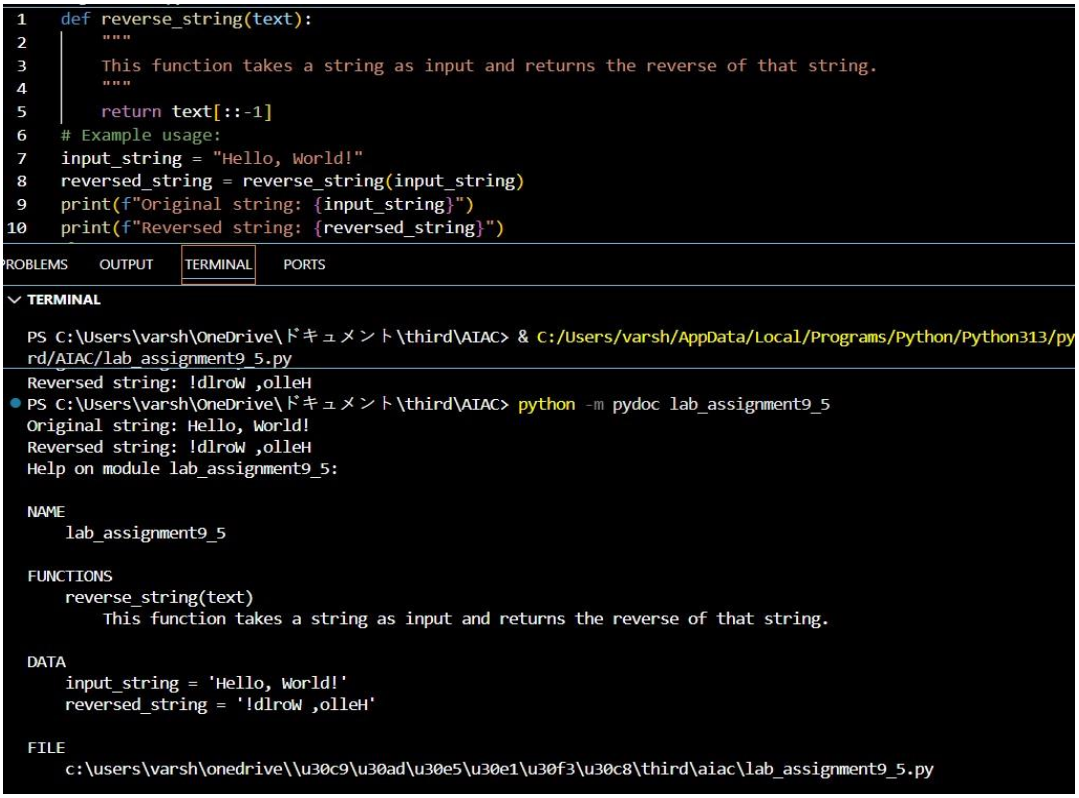
o (a) Docstring o (b) Inline

comments o (c) Google-style

documentation

2. Compare the three documentation styles.

3. Recommend the most suitable style for a utility-based string library. **Docstring**



Inline comments

```
def reverse_string(text):
    # This function takes a string as input and returns the reversed version of that string.
    return text[::-1]
input_string = "Hello, World!"
reversed_string = reverse_string(input_string)
print(f"Original string: {input_string}")
print(f"Reversed string: {reversed_string}")
```

```
OBLEMS  OUTPUT  TERMINAL  PORTS

TERMINAL
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> python -m pydoc lab_assignment9_5
DESCRIPTION
    def reverse_string(text):
        """
        This function takes a string as input and returns the reverse of that string.
        """
        return text[::-1]
    # Example usage:
    input_string = "Hello, World!"
    reversed_string = reverse_string(input_string)
    print(f"Original string: {input_string}")
    print(f"Reversed string: {reversed_string}")

FUNCTIONS
    reverse_string(text)
        # Inline documentation:

DATA
    input_string = 'Hello, World!'
    reversed_string = '!dlroW ,olleH'

FILE
```

Google style documentation

```
22  def reverse_string(text):
23      """
24      Reverses the given string.
25      Args:
26          text (str): The string to be reversed.
27      Returns:
28          str: The reversed string.
29      Raises:
30          TypeError: If the input is not a string.
31      """
32      if not isinstance(text, str):
33          raise TypeError("Input must be a string")
34      return text[::-1]
35  input_string = "Hello, World!"
36  reversed_string = reverse_string(input_string)
37  print(f"Original string: {input_string}")
38  print(f"Reversed string: {reversed_string}")
```

```
PROBLEMS  OUTPUT  TERMINAL  PORTS

TERMINAL
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> python -m pydoc lab_assignment9_5
    print(f"Reversed string: {reversed_string}")

FUNCTIONS
    reverse_string(text)
        Reverses the given string.
        Args:
            text (str): The string to be reversed.
        Returns:
            str: The reversed string.
        Raises:
            TypeError: If the input is not a string.

DATA
    input_string = 'Hello, World!'
    reversed_string = '!dlroW ,olleH'
```

Docstring – Gives a short description inside the function. It is clear and commonly used for simple explanations.

Inline comments– Very brief and placed above the code, but not structured or detailed.

Google style documentation – Well-structured with sections like Args, Returns, and Raises. It is detailed and professional.

For a utility-based string library, the Google style documentation is most suitable because it is clear, organized, and helpful for other developers.

Problem 2: Password Strength Checker

Consider the function: def

check_strength(password):

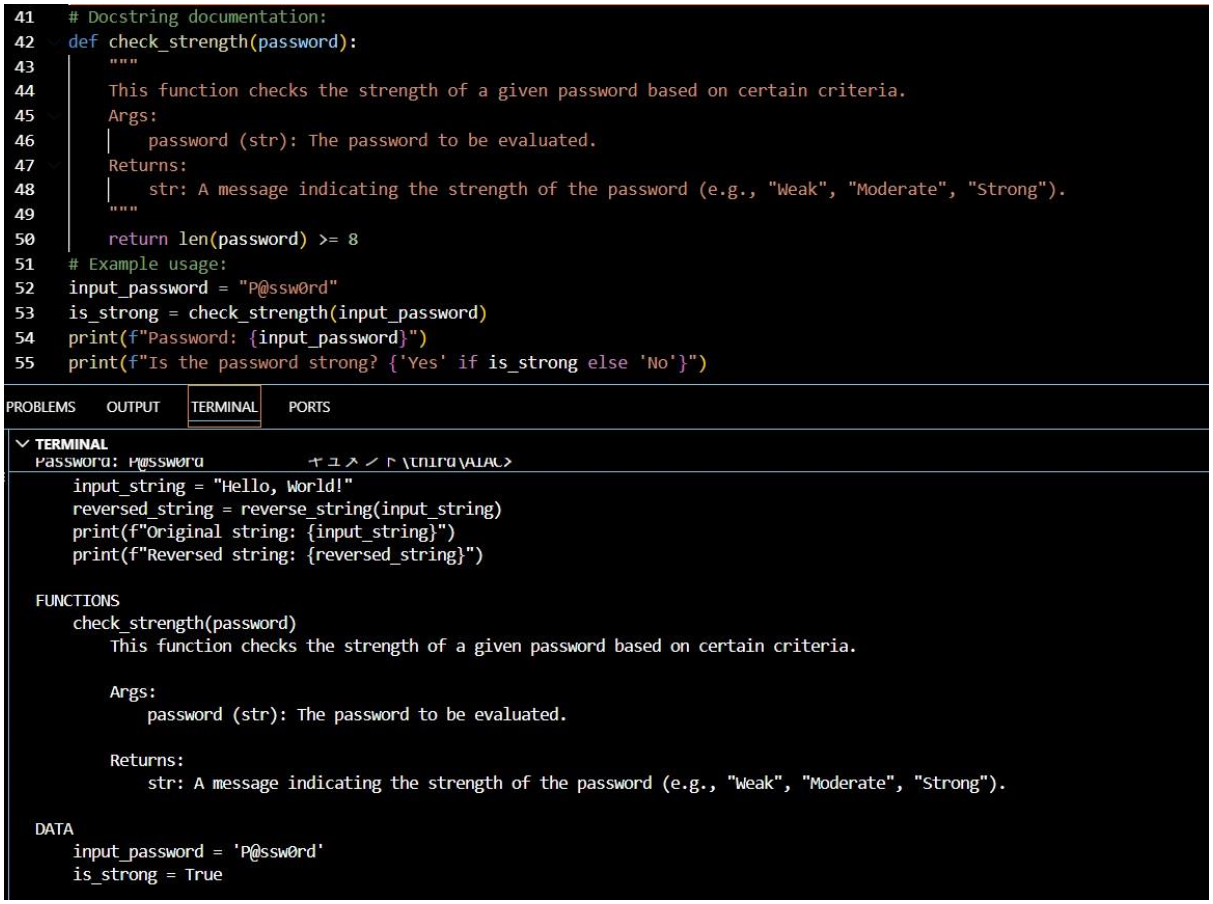
return len(password) >= 8

Task:

1. Document the function using docstring, inline comments, and Google style.

2. Compare documentation styles for security-related code.

3. Recommend the most appropriate style.



```
41  # Docstring documentation:
42  def check_strength(password):
43      """
44      This function checks the strength of a given password based on certain criteria.
45      Args:
46          password (str): The password to be evaluated.
47      Returns:
48          str: A message indicating the strength of the password (e.g., "Weak", "Moderate", "Strong").
49      """
50      return len(password) >= 8
51  # Example usage:
52  input_password = "P@ssw0rd"
53  is_strong = check_strength(input_password)
54  print(f"Password: {input_password}")
55  print(f"Is the password strong? {'Yes' if is_strong else 'No'}")
```

PROBLEMS OUTPUT TERMINAL PORTS

TERMINAL

```
Password: P@ssw0rd          ヤユメノト\third\AIAC>
    input_string = "Hello, World!"
    reversed_string = reverse_string(input_string)
    print(f"Original string: {input_string}")
    print(f"Reversed string: {reversed_string}")

FUNCTIONS
    check_strength(password)
        This function checks the strength of a given password based on certain criteria.

        Args:
            password (str): The password to be evaluated.

        Returns:
            str: A message indicating the strength of the password (e.g., "Weak", "Moderate", "Strong").

DATA
    input_password = 'P@ssw0rd'
    is_strong = True
```

```python
def check_strength(password):
    # This function checks if the password is strong based on its length.
    return len(password) >= 8
input_password = "P@ssw0rd"
is_strong = check_strength(input_password)
print(f"Password: {input_password}")
print(f"Is the password strong? {'Yes' if is_strong else 'No'}")
```

```
PROBLEMS  OUTPUT  TERMINAL  PORTS

TERMINAL

PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> python -m pydoc lab_assignment_9_5
Use help(str) for help on the str class.
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> python -m pydoc lab_assignment9_5
Password: P@ssw0rd
Is the password strong? Yes
Help on module lab_assignment9_5:

NAME
    lab_assignment9_5 - # Inline documentation:

FUNCTIONS
    check_strength(password)
        # Inline documentation:

DATA
    input_password = 'P@ssw0rd'
    is_strong = True
```

```python
def check_strength(password):
    """
    Evaluates the strength of a password based on its length.
    Args:
        password (str): The password to be evaluated.
    Returns:
        bool: True if the password is strong (length >= 8), False otherwise.
    Raises:
        TypeError: If the input is not a string.
    """
    if not isinstance(password, str):
        raise TypeError("Input must be a string")
    return len(password) >= 8
```

```
PROBLEMS  OUTPUT  TERMINAL  PORTS

TERMINAL

PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> python -m pydoc lab_assignment9_5
    check_strength(password)
        Evaluates the strength of a password based on its length.

        Args:
            password (str): The password to be evaluated.

        Returns:
            bool: True if the password is strong (length >= 8), False otherwise.
        Raises:
            TypeError: If the input is not a string.

FILE
    c:\users\varsh\onedrive\\u30c9\u30ad\u30e5\u30e1\u30f3\u30c8\third\aiac\lab_assignment9_5.py
```

Docstring style gives a clear description, arguments, and return values. It is readable but may lack strict structure for larger security projects.

Inline documentation is short and simple, but not detailed enough for security-related code where clarity is very important.

Google style documentation is more structured and detailed. It clearly explains arguments, return types, and possible errors which is important in security code.
For security-related code, Google style documentation is the most appropriate because it is clear, structured, and explains exceptions properly, reducing misunderstanding and security risks.

Problem 3: Math Utilities Module

Task:

1. Create a module math_utils.py with functions:

o square(n) o

cube(n) o

factorial(n)

2. Generate docstrings automatically using AI tools.

3. Export documentation as an HTML file.

```python
def square(n: float) -> float:
    """
    This function takes a number as input and returns its square.

    Parameters:
    n (float): The number to be squared

    Returns:
    float: The square of the input number
    """
    return n ** 2
def cube(n: float) -> float:
    """
    This function takes a number as input and returns its cube.

    Parameters:
    n (float): The number to be cubed

    Returns:
    float: The cube of the input number
    """
    return n ** 3
def factorial(n: int) -> int:
    """
    This function takes a number as input and returns its factorial.

    Parameters:
    n (int): The number to be factorialized

    Returns:
    int: The factorial of the input number
    """
    if n == 0:
        return 1
    else:
        return n * factorial(n-1)
```

```
NAME
    math_util

FUNCTIONS
    cube(n: float) -> float
        This function takes a number as input and returns its cube.

        Parameters:
        n (float): The number to be cubed

        Returns:
        float: The cube of the input number

    factorial(n: int) -> int
        This function takes a number as input and returns its factorial.

        Parameters:
        n (int): The number to be factorialized

        Returns:
        int: The factorial of the input number

    square(n: float) -> float
        This function takes a number as input and returns its square.

        Parameters:
        n (float): The number to be squared

        Returns:
        float: The square of the input number

FILE
    c:\users\varsh\onedrive\\u30c9\u30ad\u30e5\u30e1\u30f3\u30c8\third\aiac\math_util.py
```

index

math_util c:\users\varsh\onedrive\ドキュメント\third\aiac\math_util.py

Functions

cube(n: float) -> float
This function takes a number as input and returns its cube.

Parameters:
n (float): The number to be cubed

Returns:
float: The cube of the input number

factorial(n: int) -> int
This function takes a number as input and returns its factorial.

Parameters:
n (int): The number to be factorialized

Returns:
int: The factorial of the input number

square(n: float) -> float
This function takes a number as input and returns its square.

Parameters:
n (float): The number to be squared

Returns:
float: The square of the input number

Problem 4: Attendance Management Module Task:

1. Create a module attendance.py with functions:

o mark_present(student)

o mark_absent(student) o

get_attendance(student)

2. Add proper docstrings.

3. Generate and view documentation in terminal and browse

```python
def mark_present(student: str) -> None:
    """
    Marks a student as present in the attendance record.

    Args:
        student (str): The name of the student to be marked as present.
    """
    print(f"{student} is marked as present.")
def mark_absent(student: str) -> None:
    """
    Marks a student as absent in the attendance record.
    Args:
        student (str): The name of the student to be marked as absent.
    """
    print(f"{student} is marked as absent.")
def get_attendance(student: str) -> str:
    """Retrieves the attendance status of a student.
    Args:
        student (str): The name of the student whose attendance status is to be retrieved.
    Returns:
        str: The attendance status of the student (e.g., "Present", "Absent").
    """
    # For demonstration purposes, we will return a dummy attendance status.
    # In a real implementation, this would retrieve the actual attendance status from a database or record.
    return "Present"  # or "Absent" based on the actual attendance record

```

```
Help on module attendance:

NAME
    attendance

FUNCTIONS
    get_attendance(student: str) -> str
        Retrieves the attendance status of a student.
        Args:
            student (str): The name of the student whose attendance status is to be retrieved.
        Returns:
            str: The attendance status of the student (e.g., "Present", "Absent").

    mark_absent(student: str) -> None
        Marks a student as absent in the attendance record.
        Args:
            student (str): The name of the student to be marked as absent.

    mark_present(student: str) -> None
        Marks a student as present in the attendance record.

        Args:
            student (str): The name of the student to be marked as present.

FILE
    c:\users\varsh\onedrive\\u30c9\u30ad\u30e5\u30e1\u30f3\u30c8\third\aiac\attendance.py
```

index

attendance c:\users\varsh\onedrive\ドキュメント\third\aiac\attendance.py

Functions

get_attendance(student: str) -> str
Retrieves the attendance status of a student.
Args:
student (str): The name of the student whose attendance status is to be retrieved.
Returns:
str: The attendance status of the student (e.g., "Present", "Absent").

mark_absent(student: str) -> None
Marks a student as absent in the attendance record.
Args:
student (str): The name of the student to be marked as absent.

mark_present(student: str) -> None
Marks a student as present in the attendance record.

Args:
student (str): The name of the student to be marked as present.

Problem 5: File Handling Function

Consider the function: def

read_file(filename): with

open(filename, 'r') as f:

return f.read()

Task:

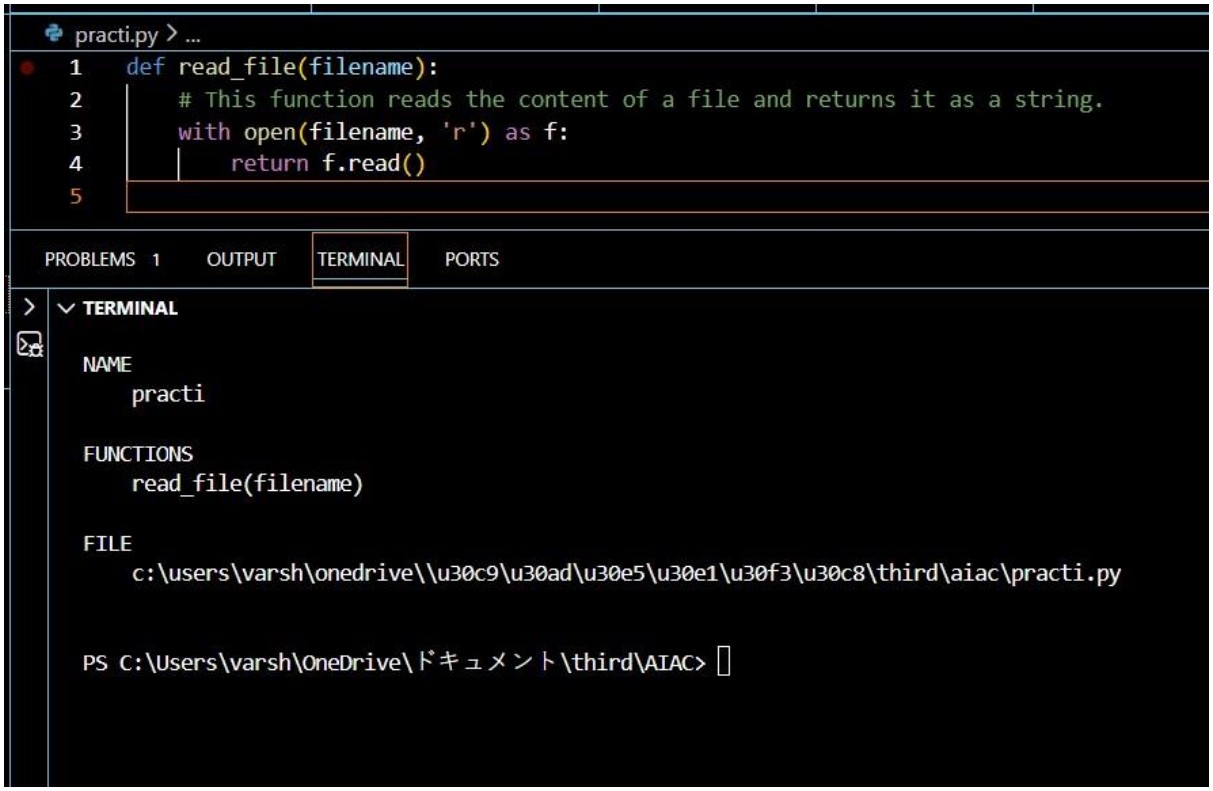
1. Write documentation using all three formats.

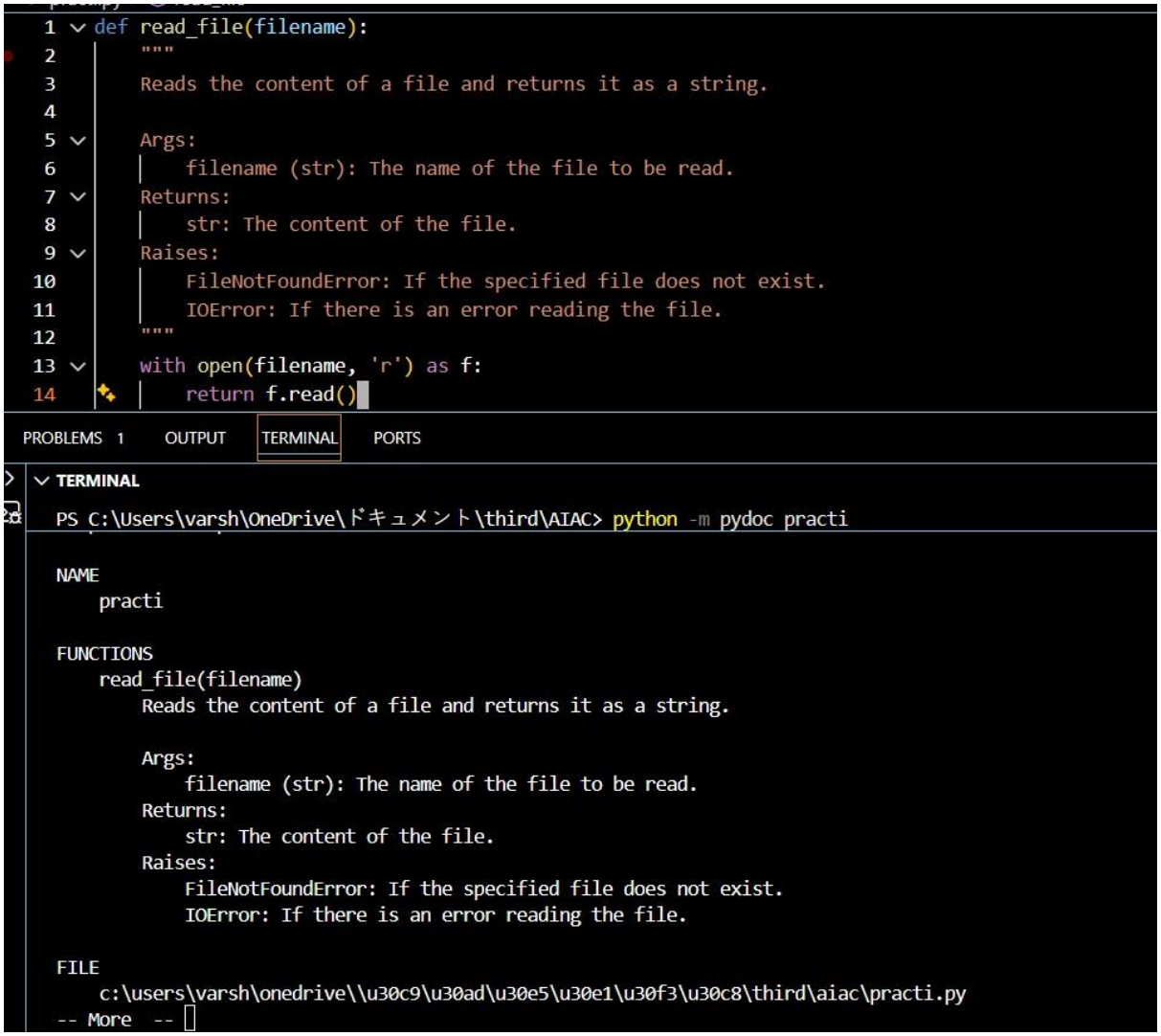
2. Identify which style best explains exception handling.

3. Justify your recommendation.

```
40  def read_file(filename):
41      """
42      This function reads the content of a file and returns it as a string.
43
44      Parameters:
45      filename (str): The name of the file to be read
46
47      Returns:
48      str: The content of the file
49      """
50      with open(filename, 'r') as f:
51          return f.read()
```

```
PROBLEMS 1   OUTPUT   TERMINAL   PORTS

TERMINAL
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> python -m pydoc math_util
FUNCTIONS
    read_file(filename)
        This function reads the content of a file and returns it as a string.

        This function reads the content of a file and returns it as a string.


        Parameters:
        filename (str): The name of the file to be read

        Returns:
        str: The content of the file

FILE
    c:\users\varsh\onedrive\\u30c9\u30ad\u30e5\u30e1\u30f3\u30c8\third\aiac\math_util.py
```


practi.py > ...
1 def read_file(filename):
2 # This function reads the content of a file and returns it as a string.
3 with open(filename, 'r') as f:
4 return f.read()
5
PROBLEMS 1
OUTPUT
TERMINAL
PORTS
TERMINAL
NAME
practi
FUNCTIONS
read_file(filename)
FILE
c:\users\varsh\onedrive\\u30c9\u30ad\u30e5\u30e1\u30f3\u30c8\third\aiac\practi.py
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>



It clearly mentions possible errors like FileNotFoundError and IOError.
This helps developers understand what can go wrong while reading a file.
Inline style does not mention exceptions, and basic docstring style does not clearly specify errors.
For reliable and safe code, especially file handling, Google style is more informative and professional.